

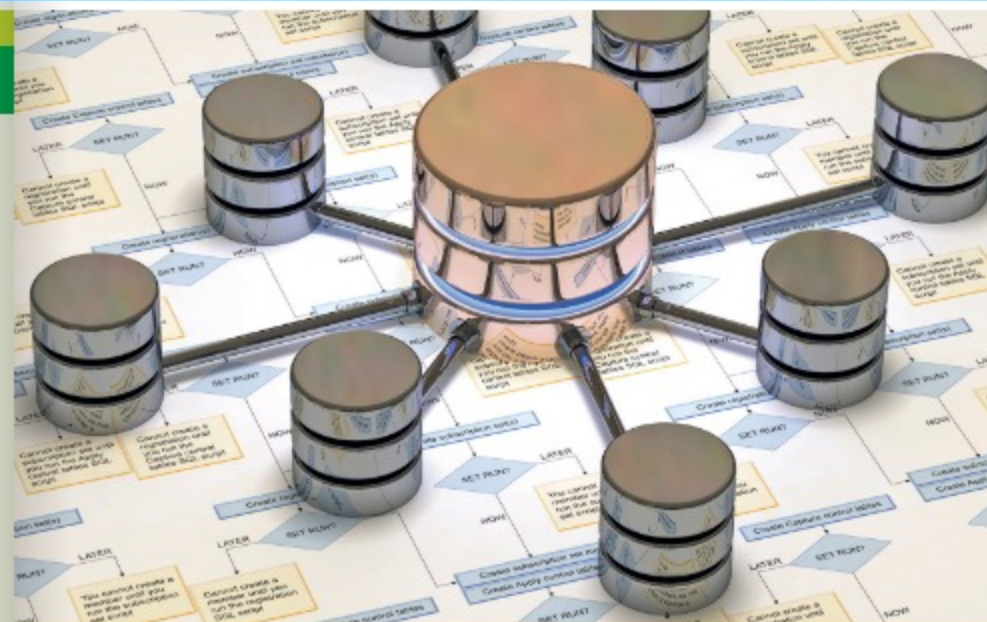
ACTIVIDADES FINALES

13. STREAM

- 13.18.** A partir de una tabla de elementos de la clase `Socio`, definida en la Actividad de aplicación 13.16, crea un `Stream` ordenado por dni y muéstralo por pantalla.
- 13.19.** Repite la Actividad de aplicación 13.18, pero en vez de mostrar los elementos del `Stream` obtén, a partir de él, una tabla de tipo `String` con los DNI de los socios.
- 13.20.** Realiza la Actividad de aplicación 13.19, pero obteniendo una lista, en vez de una tabla.
- 13.21.** Repite la Actividad de aplicación 13.18 de forma que se obtenga una colección con los nombres en orden alfabético.
- 13.22.** A partir de la misma tabla de socios de la Actividad de aplicación 13.16 calcula, usando un `Stream`, el número medio de familiares por socio.
- 13.23.** Implementa un programa en el que, a partir de dos listas de enteros ordenadas en sentido creciente, se obtenga una única lista, también ordenada, fusión de las dos anteriores. Desarrolla el algoritmo de forma no destructiva, es decir, que las listas utilizadas como parámetros de entrada se mantengan intactas.
- 13.24.** Repetir la Actividad de ampliación 12.37, pero usando `Stream` para todos los listados.

■ Actividades de ampliación

- 13.25.** Utilizando el método `reduce()` de la interfaz `Stream`, calcula el producto de los elementos de un `Stream` de enteros.
- 13.26.** Escribe los números primos menores que 100 por medio de la criba de Eratóstenes. A partir de una lista con los números del 2 al 100, eliminamos los múltiplos de los números primos menores que 10: 2, 3, 5 y 7. Utiliza `Stream` para cribar los números.
- 13.27.** Repite la Actividad resuelta 12.11, pero usando `Stream` para todos los listados.
- 13.28.** Vuelve a realizar la Actividad resuelta 12.14 usando `Stream` en los listados.
- 13.29.** Repite la Actividad de ampliación 12.37 usando `Stream` en los listados.
- 13.30.** Usando `Stream`, obtén una lista de todos los números impares, múltiplos de 3 o de 5, menores que 1000.



Conexión a base de datos: JDBC

Objetivos

- Realizar la persistencia de los datos de una aplicación mediante los servicios de un SGBD.
- Configurar el driver JDBC para poder acceder a SGBD de distintos fabricantes.
- Crear una conexión entre un programa Java y un SGBD.
- Ejecutar instrucciones de un SGBD desde una aplicación que funciona como cliente.
- Insertar los datos de una aplicación en una base de datos.
- Recuperar información almacenada en una base de datos, para su posterior uso, desde un programa Java.
- Realizar el mapeo objeto-relacional de clases simples.

Contenidos

- 14.1. API JDBC
- 14.2. Driver
- 14.3. Conexión
- 14.4. Ejecución de sentencias
- 14.5. Clase `ResultSet`
- 14.6. SQL Injection
- 14.7. Sentencias parametrizadas
- 14.8. Operaciones CRUD
- 14.9. Objeto de acceso a datos

UNIDAD 14

Introducción

La mayoría de las aplicaciones, tras recoger una serie de datos y procesarlos, generan distintos resultados en forma de nuevos datos. Todos estos datos necesitan ser almacenados de forma permanente para su posterior uso. Sin la posibilidad de que una aplicación guarde la información necesaria sería imposible llevar a cabo muchas tareas.

Entre las técnicas de persistencia hemos visto que es posible guardar datos mediante ficheros. Otra alternativa consiste en utilizar los servicios de un sistema gestor de base de datos (SGBD) para que almacene y custodie toda la información necesaria de una aplicación.

14.1. API JDBC

La API de JDBC se encuentra en el paquete `java.sql` y está compuesto por un sinnúmero de clases que trabajan de forma conjunta. No es necesario conocerlas todas, solo es imprescindible saber cuáles son las principales:

- **DriverManager**: permite manipular los distintos drivers. Con cada driver se puede acceder a un SGBD distinto.
- **Connection**: crea una conexión entre la aplicación y la base de datos.
- **Statement**: representa una sentencia SQL que ejecutará el servidor de base de datos.
- **PreparedStatement**: también representa una sentencia SQL, que permite configurar o parametrizar fácilmente valores en la consulta, como por ejemplo la edad de un alumno o su fecha de nacimiento en una condición.
- **ResultSet**: representa una tabla con el resultado que genera el SGBD tras ejecutar una sentencia de consulta de información (SELECT).

Argot técnico

API (Applications Programming Interface) es un conjunto de clases que trabajan de forma coordinada y conjunta proporcionando ciertos servicios, como por ejemplo interactuar con una base de datos, configurar una red o gestionar los movimientos de un robot.

JDBC: Java DataBase Connectivity. Es una API que permite ejecutar sentencias SQL en un SGBD desde una aplicación Java, que funcionará como un cliente que accede a los servicios del servidor de base de datos.

SGBD (sistema gestor de base de datos) es un conjunto de software que permite almacenar y gestionar información en una base de datos. También proporciona servicios de recuperación e integridad de datos, control de acceso de usuarios, copias de seguridad, etcétera.

Argot técnico

SQL (Structured Query Languages) es el lenguaje que permite interactuar con un SGBD, haciendo posible la manipulación de tablas y datos.

Recuerda

Para poder utilizar cualquiera de estas clases deberás usar su nombre cualificado. Por ejemplo, para emplear una conexión

```
java.sql.Connection
```

o importar cada clase utilizada

```
import java.sql.Connection;
```

14.2. Driver

Cada fabricante de un SGBD, al desarrollar su producto, usa mecanismos propios (protocolos, llamadas, API de bajo nivel, etc.) que establecen una conexión con la base de datos y permiten acceder a sus servicios. Las clases que componen la API de JDBC no conocen estos detalles propios de cada producto: MySQL, Oracle Database, PostgreSQL...

Por lo tanto, ¿cómo es posible que las clases de la API de JDBC finalmente lleguen a establecer conexión con el servidor de base de datos? El mecanismo es muy sencillo: entre las clases que componen la API de JDBC existe una especial (que se denomina **Driver**) que será específica de cada SGBD. De hecho, cuando un fabricante desarrolla un nuevo SGBD deberá desarrollar también el driver que permite su uso con JDBC. La clase **Driver** o driver de JDBC se añadirá a nuestro programa en función del SGBD que hayamos seleccionado.

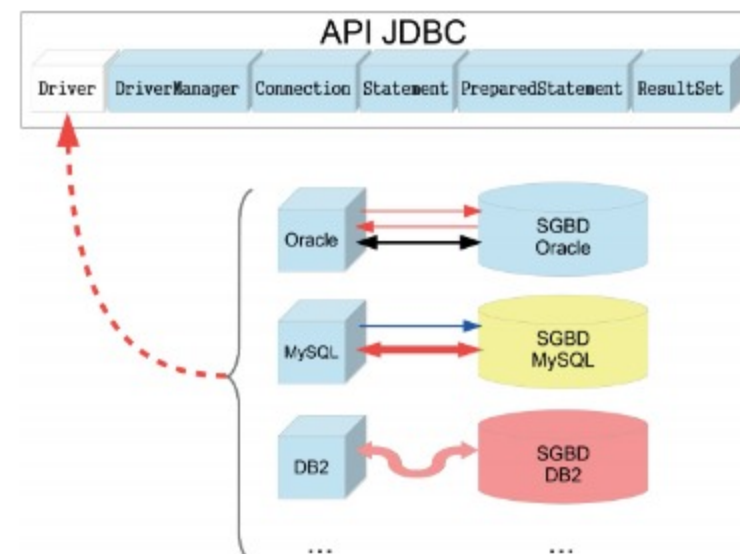


Figura 14.1. Cada fabricante desarrolla un driver JDBC que conoce las peculiaridades del SGBD asociado, haciendo de puente o traductor entre el resto de las clases de la API de JDBC y el servidor de base de datos. Las distintas flechas de colores entre el driver y el SGBD representan los detalles propios de la comunicación.

De esta forma, el driver permite que cualquier aplicación Java, a través de la API JDBC sea capaz de utilizar los servicios de cualquier SGBD para el que exista un driver JDBC. Y así se evita que cada aplicación Java tenga que conocer los detalles internos propios de cada SGBD.

Recuerda

MariaDB es un clon de MySQL que mantiene exacta la interfaz externa y realiza una continua mejora de los procedimientos y algoritmos, y corrige errores. Por lo tanto, el driver de MySQL sirve también para MariaDB.

La manera de añadir un driver a nuestro proyecto consiste en insertar la biblioteca que contiene la clase correspondiente al driver. La forma más sencilla es usar el navegador de proyectos de NetBeans y pulsar el botón derecho del ratón sobre *Libraries* en nuestro proyecto (véase Figura 14.2), y seleccionar en el menú contextual *Add JAR/Folder*.

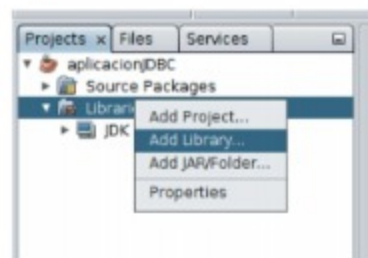


Figura 14.2. Mecanismo para insertar las librerías que necesitamos en nuestro proyecto.

Recuerda

Una biblioteca no es más que una serie de clases (API) encapsuladas en un fichero.

Ahora solo queda seleccionar el fichero .jar que incluye el driver adecuado. Dicho fichero tendrá que descargarse de la web del fabricante del SGBD.

Driver preinstalado en el IDE

Algunos drivers, como el de PostgreSQL, vienen preinstalados por defecto en NetBeans. Incluso las versiones antiguas de NetBeans tenían el driver de MySQL (en las versiones recientes no se incluyen).

Recuerda

En ocasiones, dependiendo del sistema operativo, la versión del IDE, del SGBD y la versión del driver, es posible que exista alguna incompatibilidad. En caso de que la aplicación no funcione, se recomienda probar una versión distinta del driver.

Si el driver que se va a usar viene con NetBeans, no es necesario descargar nada y tan solo hay que seleccionarlo de la forma: botón derecho sobre *Libraries* en nuestro proyecto y elegimos *Add Library* en el menú contextual. A continuación seleccionaremos el driver que nos interese (o cualquier otra librería que nos haga falta). El único problema es que la versión del driver no será la última.

En la Figura 14.3 se aprecian los drivers de MySQL (que se incluían en versiones antiguas de NetBeans) y PostgreSQL.

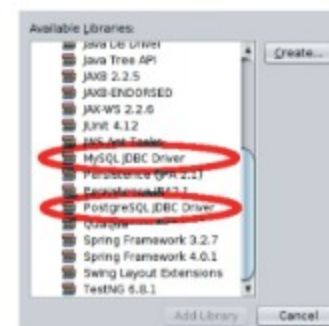


Figura 14.3. Ventana de selección de una nueva biblioteca. En este caso se aprecia que disponemos del driver JDBC de MySQL y de PostgreSQL.

Añadir otro driver

La opción *Add JAR/Folder* permite añadir una biblioteca al proyecto que previamente habremos descargado en nuestro equipo. Esta opción posibilita añadir drivers de otros SGBD o modificar la versión de los drivers preinstalados.

Si deseamos utilizar la última versión del driver de MySQL (que denomina a su driver como *Connector/J*), solo tendremos que buscar y descargarlo de su web.

14.3. Conexión

Antes de trabajar con la base de datos, hay que crear una conexión entre nuestra aplicación y el SGBD. Esta conexión funciona como un tubo que comunica ambas partes, permitiendo que las sentencias SQL viajen desde la aplicación al SGBD y los resultados de las consultas se muevan en sentido contrario. Mientras la necesitamos, la conexión deberá permanecer abierta; una vez que ya no sea útil, hay que cerrarla. En el caso de que una conexión no se cierre, quedará abierta consumiendo recursos del SGBD.

Para crear una conexión disponemos del método estático de `DriverManager`:

```
■ Connection getConnection(String url, String usuario, String password)
```

El primer parámetro identifica la base de datos a la que queremos acceder, en general, para MySQL tendrá la forma:

```
jdbc:mysql://<servidor>/<base de datos>
```

Donde:

- <servidor>: es el nombre o la dirección IP de la máquina donde está instalado el servidor de BD. En el caso que el servidor esté instalado en la máquina local, puede usarse *localhost*.
- <base de datos>: nombre de la BD dentro del SGBD.

Argot técnico

URL (Uniform Resource Locator) es un identificador de recursos con un formato establecido. El formato general es:
esquema://máquina/recurso

Existen sistemas operativos y servidores de BD que no son sensibles a mayúsculas y minúsculas, es decir, no distinguen entre las palabras «hola» y «HOLA». Sin embargo, los sistemas operativos de la familia GNU/Linux sí lo hacen. Por lo tanto, se recomienda escribir siempre los nombres de bases de datos, tablas y campos de forma idéntica a como se crearon.

Recuerda

El uso de una contraseña sin cifrar en JDBC no es muy seguro, ya que esta viajará por la red desde nuestra aplicación hasta el servidor sin ningún tipo de seguridad.

Para evitar esto es necesario enviar la contraseña cifrada en lugar de como texto «en claro».

Los mecanismos para cifrar la contraseña son ajenos a JDBC, por lo tanto, para no complicar esta unidad usaremos contraseñas sin cifrar.

Los siguientes dos parámetros son una cadena con el nombre del usuario y su contraseña.

El método devuelve un objeto de tipo `Connection`, que representa la conexión establecida entre nuestra aplicación y la base de datos.



Figura 14.4. Cada objeto de la clase `Connection` representa una conexión entre nuestra aplicación y la BD.

Suponiendo que nuestra base de datos se llama «Instituto» y que disponemos del usuario «Pepe» con contraseña «12345», la manera de crear una conexión será:

```
Connection con;
String url = "jdbc:mysql://localhost/Instituto";
con = DriverManager.getConnection(url, "Pepe", "12345");
```

Cada conexión es una tubería que permite que viajen distintos objetos. Cada tipo de objetos representa una información útil para el SGBD o para la aplicación (véase Figura 14.5).

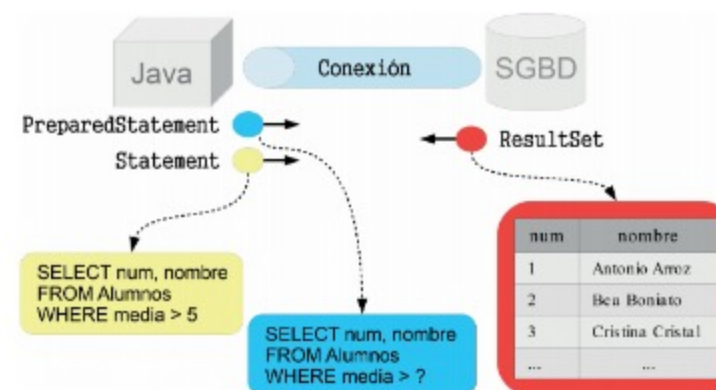


Figura 14.5. Tipos de objetos que viajan por una conexión, el sentido en el que lo hacen y la información que representan.

El método `getConnection()`, en el caso que se produzca algún error al crear la conexión, lanzará alguna de las siguientes excepciones:

- `SQLException`: si ocurre algún error en el acceso a la base de datos o la URL es `null`.
- `SQLException`: cuando el tiempo que ha transcurrido sin llegar a conectar a la base de datos es excesivo.

Recuerda

Cuando usamos métodos que lanzan excepciones (checked) necesitamos añadir el código pertinente para manejarlas, ya sea añadiendo una cláusula `throws` al método que contiene el código o utilizando bloques `try-catch`.

Para mantener una mayor legibilidad de los ejemplos hemos decidido omitir el código que maneja las excepciones.

Cuando no necesitemos una conexión, es necesario cerrarla mediante el método:

- `void close()`

14.4. Ejecución de sentencias

Para ejecutar una sentencia SQL (SELECT, INSERT, UPDATE o DELETE) tendremos que utilizar un objeto de la clase `Statement`. Estos objetos, al igual que ocurre con la conexión, no se crean mediante el operador `new`, sino mediante métodos que se encargan de construir, configurar y devolver los objetos que necesitamos.

Recuerda



La sintaxis simplificada de la sentencia SELECT es:

```
SELECT <lista de campos> | *
FROM <tabla> | <join>
[WHERE <condición>]
[ORDER BY <lista de campos>]
```

La sentencia SELECT tiene una sintaxis demasiado compleja para representarla aquí. En caso de tener que utilizar subconsultas, funciones de agregados o agrupamientos, se recomienda acceder a la documentación de SQL.

Una de las sintaxis de INSERT es:

```
INSERT INTO <tabla> [(<lista de campos>)]
VALUES (<valor 1>, <valor 2>, ...)
```

La sintaxis de UPDATE es:

```
UPDATE <tabla>
SET <campo1> = <valor1>,
    <campo2> = <valor2>
...
[WHERE <condición>]
```

La sintaxis de DELETE es:

```
DELETE
FROM <tabla>
[WHERE <condición>]
```

Las sintaxis mostradas son las más utilizadas, pero existen otros operadores y alternativas.

Lo primero que haremos será crear un objeto de tipo `Statement`:

```
Statement sentencia = con.createStatement();
```

El objeto `con` es de tipo `Connection` y representa la conexión creada entre la aplicación y el SGBD. Disponemos de dos métodos para ejecutar sentencias SQL. El primero está reservado a las consultas (SELECT) que devuelven datos con el resultado proporcionado por el SGBD. El segundo método está reservado para las sentencias INSERT, UPDATE o DELETE que no devuelven resultados.

14.4.1. Ejecución de consultas (SELECT)

El método `executeQuery()` de `Statement` ejecuta una consulta y devuelve el resultado de esta mediante un objeto de tipo `ResultSet`.

Su prototipo es:

```
ResultSet executeQuery(String sql)
```

Veamos cómo consultar la tabla Alumnos:

```
String sql = "SELECT * FROM Alumnos";
Statement sentencia = con.createStatement();
ResultSet rs = sentencia.executeQuery(sql);
```

Una vez que obtenemos los resultados de la consulta (englobados en un objeto de tipo `ResultSet`), queda trabajar con ellos. Esto se verá en un apartado posterior.

Al igual que la mayoría de las clases que componen la API de JDBC, el método `executeQuery()` lanza, en el caso de que exista algún problema, dos excepciones:

- `SQLException`.
- `SQLTimeoutException`.

Recuerda



El script que crea e inserta la base de datos Instituto en MySQL está disponible en la web de la Editorial Paraninfo como recurso digital.

14.4.2. Ejecución de sentencias INSERT, UPDATE o DELETE

En este caso, la ejecución de cualquiera de estas sentencias no devuelve un conjunto de datos como resultados, sino simplemente realizan la operación indicada en el servidor. Disponemos del método:

- `int executeUpdate(String sql)`: ejecuta la sentencia `sql` correspondiente y devuelve el número de filas que han sido afectadas por la sentencia, es decir, el número de registros que se han eliminado, actualizado o insertado.

Que el nombre del método no nos lleve a engaño; este sirve tanto para insertar y actualizar como para eliminar registros en una tabla.

Actividad resuelta 14.1

Incrementar la nota media en un punto a todos los alumnos del curso «1B». Usar una consulta SQL que haga uso de `UPDATE`.

Solución

Para resolver esta actividad, tras crear un nuevo proyecto de NetBeans, el primer paso será añadir el driver JDBC de MySQL. El código que lleva a cabo esto es:

```
Connection con; //declaramos las variables necesarias
Statement sentencia;
String sql;
```

```
String url = "jdbc:mysql://localhost/Instituto";
try {
    con = DriverManager.getConnection(url, "Pepe", "12345");
    sentencia = con.createStatement();
    String sql = "UPDATE Alumnos SET media=media+1 * +
                *WHERE curso = '1B'"; //el curso es de tipo VARCHAR, por lo
    //tanto, necesita que sus valores se entrecomillen
    sentencia.executeUpdate(sql);
    con.close(); //cerramos la conexión
    System.out.println("Se ha modificado la nota media.");
} catch (SQLException ex) {
    System.out.println("Ha ocurrido algún error.");
}
```

Antes de poder ejecutar este código es imprescindible que:

1. La base de datos Instituto esté creada en el servidor.
2. Que exista el usuario Pepe (con clave 12345) y tenga los permisos necesarios en la base de datos Instituto.

Recuerda



El uso de las clases que forman la API de JDBC requieren de las importaciones oportunas. Conforme se escriben el código, el propio IDE nos facilita la opción de realizar de forma automática la importación de estas clases.

Es necesario tener mucho cuidado al elegir la clase que se va a importar, ya que existen varias clases con idéntico nombre pero en distintos paquetes. Hemos de elegir siempre las clases del paquete `java.sql`.

Para comprobar si nuestra aplicación está realizando las actualizaciones oportunas es necesario acceder al SGBD (mediante un terminal o algún cliente gráfico) y revisar que la nota media de los alumnos de 1B está sufriendo un incremento.

Nota técnica



En las sentencias SQL que manejamos desde Java, en ocasiones, es necesario utilizar comillas para algunos literales cadena o fecha.

Los literales cadena en Java utilizan comillas dobles `"`, por lo tanto, dejamos las comillas simples `'` para los literales usados en SQL. También es posible usar las comillas dobles mediante la secuencia de escape: `\`.

La sentencia UPDATE de la Actividad resuelta 14.1 se podría haber escrito de cualquiera de estas dos maneras:

- `String sql = "UPDATE Alumnos SET media=media+1 WHERE curso = '1B'";`
- `String sql = "UPDATE Alumnos SET media=media+1 WHERE curso = \"1B\"";`

Las comillas en rojo `"` son las que se usan en las cadenas de Java, y el resto de comillas (en azul) son las utilizadas por las sentencias SQL.

Actividad resuelta 14.2

Realizar una aplicación que solicite todos los datos de un nuevo alumno y los inserte en la base de datos.

Solución

```
Connection con;
Statement sentencia;
String sql;
```

```
String url = "jdbc:mysql://localhost/Instituto";
try {
    con = DriverManager.getConnection(url, "Pepe", "12345");
    sentencia = con.createStatement();
    //vamos a solicitar todos los datos del nuevo alumno:
    System.out.println("Número de alumno:");
    int num = new Scanner(System.in).nextInt();
    System.out.println("Nombre:");
    String nombre = new Scanner(System.in).nextLine();
    System.out.println("Fecha de nacimiento:");
    String fnac = new Scanner(System.in).nextLine();
    System.out.println("Media:");
    Double media;
    media = new Scanner(System.in).useLocale(Locale.US).nextDouble();
    System.out.println("Curso:");
    String curso = new Scanner(System.in).nextLine();

    //Ahora hemos de construir la sentencia INSERT a partir de todos
    //los valores anteriores
    sql = "INSERT INTO Alumnos (num, nombre, fnac, media, curso) * +
        *VALUES (" + num + ", " + nombre + ", " + fnac + ", " +
        media + ", " + curso + ")";
    sentencia.executeUpdate(sql);
    con.close(); //cerramos la conexión
    System.out.println("Se ha insertado el nuevo alumno.");
} catch (SQLException ex) {
    System.out.println("Ha ocurrido algún error.");
}
```

En esta actividad no se realiza ninguna comprobación, por lo tanto, si el número del alumno (que es la clave) se encontrase repetido, se produciría un error.

Es interesante apreciar que el tipo de las variables no es relevante, ya que solo se usan para formar la cadena que ejecutará el SGBD. Por ello, hubiera dado igual declarar la variable `media` de tipo `Double` o de tipo `String`.

Actividad resuelta 14.3

Solicita el número de un alumno y elimínalo de la base de datos.

Solución

```
Connection con;
Statement sentencia;
String sql;
```

```
String url = "jdbc:mysql://localhost/Instituto";
try {
    con = DriverManager.getConnection(url, "Pepe", "12345");
    sentencia = con.createStatement();
    //vamos a solicitar el número del alumno a borrar:
    System.out.println("Número de alumno:");
    int num = new Scanner(System.in).nextInt();

    //Ahora construimos la sentencia DELETE:
    sql = "DELETE FROM Alumnos WHERE num = " + num;
    sentencia.executeUpdate(sql);
    System.out.println("Se ha eliminado el alumno con número: " + num);
    con.close(); //cerramos la conexión
} catch (SQLException ex) {
    System.out.println("Ha ocurrido algún error.");
}
```

14.5. Clase ResultSet

Cuando se ejecuta una consulta (SELECT) obtenemos los resultados encapsulados en un objeto de tipo `ResultSet`, que representa una tabla con los datos que genera la consulta.

Por ejemplo, si necesitamos consultar el número, nombre y nota media de los alumnos que hayan nacido después del 1 de enero de 2009, ejecutaremos el código:

```
String sql = "SELECT num, nombre, media FROM Alumnos WHERE fnac > '2009-01-01'";
Statement sentencia = con.createStatement();
ResultSet rs = sentencia.executeQuery(sql);
```

El objeto `rs` contiene los datos obtenidos de esta consulta:

num	nombre	media
1	Antonio Arroz	5.10
2	Bea Boniato	6.75
3	Cristina Cristal	7.23
4	David Dado	8.50

Figura 14.6. Listado de alumnos nacidos después del 1 de enero de 2009.

La clase `ResultSet` tiene una forma de trabajar muy parecida a un iterador. Dispone de un cursor que apunta en cada momento a una única fila (llamada *fila activa*). En la Figura 14.7 el cursor se representa mediante una flecha roja. Este sistema solo permite acceder a los datos de la fila activa, para trabajar con todos los datos del objeto `ResultSet` tendremos que ir moviendo el cursor de fila en fila.

Justo en el momento en el que se crea un objeto `ResultSet` el cursor se encuentra delante de la primera fila (Figura 14.7). Por lo tanto, hay que realizar un primer avance del cursor para colocarlo en la primera fila (Figura 14.8) y comenzar a trabajar con los datos de esta.



num	nombre	media
1	Antonio Arroz	5.10
2	Bea Boniato	6.75
3	Cristina Cristal	7.23
4	David Dado	8.50

Figura 14.7. Posición inicial de un `ResultSet`, donde aún no existe ninguna fila activa.

La forma de pasar de una fila a la siguiente se lleva a cabo mediante el método:

- `boolean next()`: mueve el cursor a la siguiente fila. Devuelve verdadero o falso si ha sido posible realizar el movimiento. Dicho en otras palabras, devuelve `false` cuando el cursor termina de recorrer todas las filas de datos y `true` en caso contrario.

El booleano que devuelve `next()` es fundamental para conocer cuándo hemos terminado de procesar todas las filas de un `ResultSet`. Es habitual utilizar `rs.next()` como la condición de un bucle `while`.



1	Antonio Arroz	5.10
2	Bea Boniato	6.75
3	Cristina Cristal	7.23
4	David Dado	8.50

Figura 14.8. Tras ejecutar el primer `next()`, el cursor apunta a la primera fila, es decir, se activa la fila correspondiente al alumno Antonio Arroz.

Ahora podemos extraer los datos de la fila activa. Para ello disponemos de los métodos:

- `getString(String nombreCampo)`: devuelve el valor del campo como una cadena.
- `int getInt(String nombreCampo)`: devuelve el valor del campo como un entero.
- `Double getDouble(String nombreCampo)`: devuelve el valor del campo como un real.
- `Date getDate(String nombreCampo)`: devuelve el valor del campo como una fecha.

Existe un método prácticamente para cada tipo de dato disponible en Java.

Ahora podemos extraer los datos de la fila activa. Por ejemplo:

```
//mostrará "Número de alumno: 1"
System.out.println("Número del alumno: " + rs.getInt("num"));
//mostrará "Nombre Antonio Arroz"
System.out.println("Nombre: " + rs.getString("nombre"));
Double nota = rs.getDouble("media"); //la variable nota será igual a 5.10
```

Todos los métodos para extraer los datos de una tabla están sobrecargados para que, en lugar de especificar el nombre del campo, se pueda indicar su posición en la consulta. Hay que tener en cuenta que `ResultSet` numera los campos de una consulta comenzando en 1.

El código anterior sería similar a:

```
//El campo 1 corresponde con "num".
System.out.println("Número del alumno: " + rs.getInt(1));
//El campo 2 corresponde con "nombre"
System.out.println("Nombre: " + rs.getString(2));
//El campo 3 corresponde con "media"
double nota = rs.getDouble(3);
```

Si de nuevo ejecutamos `rs.next()`, obtendremos `true`, ya que es posible activar la siguiente fila (Figura 14.9) y podríamos extraer de nuevo los datos de la fila activa, que ahora corresponderán a Bea Boniato.



num	nombre	media
1	Antonio Arroz	5.10
2	Bea Boniato	6.75
3	Cristina Cristal	7.23
4	David Dado	8.50

Figura 14.9. Estado del objeto `ResultSet` tras activar la siguiente fila. En este caso, la fila activa corresponde a los datos de Bea Boniato.

Podemos extraer los datos de las filas y ejecutar repetidas veces `rs.next()`, pero llegará un momento en el que hayamos visitado todas las filas del `ResultSet`, y `rs.next()` devolverá falso, indicando que no existen más filas que activar (Figura 14.10); en ese momento el cursor se coloca justo detrás de la última fila.



num	nombre	media
1	Antonio Arroz	5.10
2	Bea Boniato	6.75
3	Cristina Cristal	7.23
4	David Dado	8.50

Figura 14.10. Estado del `ResultSet` tras haber activado todas las filas. El último `rs.next()` devolverá falso y coloca el cursor justo detrás de la última fila.

La estructura habitual para trabajar con un `ResultSet` es:

```
ResultSet rs = sentencia.executeQuery(sql);
while (rs.next()) {
    //extraemos los datos de la fila activa
```

```
variable1 = rs.getString("nombre");
variable2 = rs.getInt("num");
...
}
```

Al salir del bucle `while` tenemos la certeza de que hemos recorrido todas las filas obtenidas en la consulta.

Actividad resuelta 14.4

Mostrar el nombre y fecha de nacimiento de todos los alumnos de un curso, que se solicitará al usuario por teclado.

Solución

```
Connection con;
Statement sentencia;
ResultSet rs;
String sql;

String url = "jdbc:mysql://localhost/Instituto";
try {
    con = DriverManager.getConnection(url, "Pepe", "12345");
    //vamos a solicitar el curso:
    System.out.println("Escriba un curso:");
    String curso = new Scanner(System.in).nextLine();
    sentencia = con.createStatement();
    sql = "SELECT nombre, fnac FROM Alumnos WHERE curso = '" + curso + "'";
    rs = sentencia.executeQuery(sql);
    System.out.println("Lista de alumnos:");
    while (rs.next()) {
        String nombre = rs.getString("nombre");
        Date fecha = rs.getDate("fnac");
        System.out.println("Alumno: " + nombre + "\tF. nacimiento: " + fecha);
    }
    con.close(); //cerramos la conexión
} catch (SQLException ex) {
    System.out.println("Ha ocurrido algún error.");
}
```

14.5.1. Tipos de ResultSet

El objeto `ResultSet` que hemos utilizado puede denominarse *por defecto*, ya que es de solo lectura (se pueden extraer los datos del objeto `ResultSet`, pero no modificarlos) y su cursor siempre avanza hacia delante. Es posible, utilizar otros tipos de `ResultSet` que permiten la modificación de sus datos y poder mover el cursor hacia delante o atrás, así como posicionarlo en cualquier fila.

Para obtener otros tipos de `ResultSet` es necesario crear los objetos `Statement` mediante el método sobrecargado de `Connection`:

```
Statement createStatement(int tipoResultSet, int concurrencia)
```

El parámetro `tipoResultSet` admite cualquier de las constantes:

- `ResultSet.TYPE_FORWARD_ONLY`: indica que el cursor solo podrá moverse hacia delante.
- `ResultSet.TYPE_SCROLL_INSENSITIVE`: el cursor podrá desplazarse hacia delante o atrás. Los datos contenidos en el `ResultSet` son una copia de los datos almacenados en la base de datos, por lo tanto, el `ResultSet` no es sensible a los posibles cambios que se produzcan en la base de datos.
- `ResultSet.TYPE_SCROLL_SENSITIVE`: el cursor podrá desplazarse hacia delante o atrás. El `ResultSet` es sensible a los cambios que ocurran en la base de datos. Cualquier modificación de los datos que contiene el `ResultSet` en la base de datos produce una modificación en los datos del `ResultSet`. En este caso el `ResultSet` actúa de una forma similar a una vista.

El segundo parámetro indica la posibilidad de modificar los datos contenidos en el objeto `ResultSet`:

- `ResultSet.CONCUR_READ_ONLY`: los datos contenidos en el `ResultSet` serán de solo lectura.
- `ResultSet.CONCUR_UPDATABLE`: desde la aplicación es posible modificar los datos contenidos en el objeto `ResultSet`.

Cuando usamos el método `createStatement()`, sin parámetros, los objetos `ResultSet` obtenidos por defecto serán de tipo `TYPE_FORWARD_ONLY` y `CONCUR_READ_ONLY`.

14.5.2. Métodos para mover el cursor

Existen métodos que mueven y posicionan el cursor a nuestro antojo a través de las filas de un `ResultSet`. Estos métodos solo son aplicables en objetos `ResultSet` cuyo tipo permite el desplazamiento del cursor (`TYPE_SCROLL_INSENSITIVE` y `TYPE_SCROLL_SENSITIVE`). Si se intenta mover el cursor en un `ResultSet` de tipo `TYPE_FORWARD_ONLY`, se lanzará una excepción; con este tipo de `ResultSet` solo es posible avanzar el cursor hacia la fila siguiente.

Todos los métodos que mueven el cursor devuelven un booleano que indica si ha sido posible desplazar el cursor (`true`) o, por el contrario, el movimiento del cursor no puede realizarse (en este caso devuelven `false`). Un movimiento del cursor no podrá llevarse a cabo si la fila especificada no existe o el tipo de cursor no permite ese tipo de movimiento. Estos métodos son:

- `boolean next()`: avanza el cursor y activa la siguiente fila.
- `boolean previous()`: retrocede el cursor y activa la fila anterior.
- `boolean first()`: coloca el cursor en la primera fila, activándola.
- `boolean last()`: coloca el cursor en la última fila, activándola.
- `boolean absolute(int numeroFila)`: mueve el cursor a la enésima fila del `ResultSet`. El parámetro `numeroFila` especifica el número de fila al que hay que

mover el cursor. Las filas se numeran comenzando en 1, es decir, la primera fila es la 1, la segunda la 2, etc. Existe otra forma de numerar las filas y consiste en utilizar números negativos, en este caso, la cuenta comienza por la última fila. La fila -1 es la última, la fila -2 es la penúltima, etcétera.

- `boolean relative(int cuantasFilas)`: mueve el cursor tomando como base su posición actual. El parámetro `cuantasFilas` especifica cuántas filas hay que desplazar el cursor. Si es positivo, el cursor avanza tantas filas como se especifique; y si es negativo, el cursor retrocede. Si el parámetro `cuantasFilas` es 0, el cursor no se moverá.
- `void beforeFirst()`: coloca el cursor justo delante de la primera fila (posición inicial del cursor al crear un objeto `ResultSet`). Este método no tiene ningún efecto en un cursor vacío (sin ninguna fila).
- `void afterLast()`: coloca el cursor justo después de la última fila. Esta posición del cursor es la misma que conseguimos tras recorrerlo mediante: `while(rs.next())`. Este método no hace nada si el `ResultSet` está vacío.

Actividad resuelta 14.5

Modificar la Actividad resuelta 14.4 para que los alumnos se muestren en orden inverso.

Si existiera algún criterio de ordenación en la consulta SQL, se podría modificar para que fuera justo el contrario. Pero en la Actividad resuelta 14.4 no existe criterio de ordenación alguno, por lo tanto, tendremos que colocar el cursor detrás de la última fila y recorrerlo de abajo arriba (con el método `previous()`).

Solución

```
... //el código anterior es idéntico a la actividad resuelta 14.4
//con la salvedad en la forma de crear el objeto sentencia (en rojo)
sentencia = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                                ResultSet.CONCUR_READ_ONLY);

rs = sentencia.executeQuery(sql);
rs.afterLast(); //colocamos el cursor detrás de la última fila
System.out.println("Lista de alumnos:");
while (rs.previous()) { //recorremos el ResultSet de abajo hacia arriba
    String nombre = rs.getString("nombre");
    Date fecha = rs.getDate("fnac");
    System.out.println("Alumno: " + nombre + "\tF. nacimiento: " + fecha);
}
... // falta cerrar la conexión y los bloques try-catch
```

14.5.3. Ubicación del cursor

La posibilidad de mover el cursor puede provocar que perdamos la noción de dónde se encuentra. Por ello, existe un conjunto de métodos que permiten sondear si el cursor se halla en algunas posiciones establecidas. Estos métodos preguntan: ¿está el cursor en cierta posición? Todos devuelven un booleano para indicar sí o no.

- `boolean isBeforeFirst()`: especifica si el cursor se encuentra delante de la primera fila (es la posición inicial cuando se crea un objeto `ResultSet`).
- `boolean isAfterLast()`: indica si el cursor está colocado justo detrás de la última fila. Esta posición es la que se alcanza tras recorrer el `ResultSet` mediante `while(rs.next())`.
- `boolean isFirst()`: devuelve `true` si el cursor está apuntando a la primera fila.
- `boolean isLast()`: devuelve `true` si el cursor apunta a la última fila.

Actividad resuelta 14.6

Mostrar el nombre y nota de todos los alumnos y, *a posteriori*, mostrar al alumno con la mejor nota y al alumno con la peor nota media.

Obtener a los alumnos con la mejor y peor nota es posible mediante una consulta SQL. Existen distintas soluciones: operador UNION, subconsultas, etc. En lugar de esto aprovecharemos la consulta que devuelve a todos los alumnos ordenados por su nota media en orden decreciente, ya que el primer alumno será el que posea la mejor nota y el último el que tenga la nota más baja. Por sencillez, supondremos que no existen alumnos con notas repetidas.

Solución

```
Connection con;
Statement sentencia;
ResultSet rs;
String sql;

String url = "jdbc:mysql://localhost/Instituto";
try {
    con = DriverManager.getConnection(url, "Pepe", "12345");
    sentencia = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                                     ResultSet.CONCUR_READ_ONLY);
    //vamos a consultar todos los alumnos ordenados por su nota
    sql = "SELECT nombre, media FROM Alumnos ORDER BY media DESC";
    rs = sentencia.executeQuery(sql);
    System.out.println("Todos los alumnos:");
    while (rs.next()) {
        System.out.println(rs.getString(1) + " - " + rs.getInt(2));
    }
    //El alumno con la mejor nota estará en la primera fila.
    //Y el alumno con la nota media más baja estará en la última fila.
    if (rs.first()) { //si nos podemos colocar en la primera fila
        String nombre = rs.getString("nombre");
        Double nota = rs.getDouble("media");
        System.out.println("Nota más alta: " + nombre + "\t" + nota);
    }
    if (rs.last()) { //si nos podemos colocar en la última fila
        String nombre = rs.getString("nombre");
        Double nota = rs.getDouble("media");
        System.out.println("Nota más baja: " + nombre + "\t" + nota);
    }
}
```

```
con.close(); //cerramos la conexión
} catch (SQLException ex) {
    System.out.println("Ha ocurrido algún error.");
}
```

14.6. SQL Injection

SQL Injection es una técnica de hacking muy conocida, sencilla y fácil de utilizar. Consiste en inyectar código SQL en una consulta mediante la entrada de datos. Veamos un ejemplo: suponemos que tenemos una aplicación que, en un momento dado, nos pide el nombre de un alumno para eliminarlo.

El código que utilizaremos tendrá el aspecto:

```
String nombre;
nombre = new Scanner(System.in).nextLine(); //pedimos el nombre al usuario
String sql = "DELETE FROM Alumnos WHERE nombre = '" + nombre + "'";
```

En el código hemos coloreado las comillas simples (usadas por la consulta SQL) para ganar en legibilidad.

Si el usuario introduce como nombre «Antonio Arroz», la consulta que se ejecutará en el servidor de base de datos, será:

```
DELETE FROM Alumnos
WHERE nombre = 'Antonio Arroz'
```

lo que provocaría que se eliminara a dicho alumno.

Pero si el usuario tiene conocimientos de SQL e introduce de forma malintencionada código SQL como si fuera el nombre del alumno, este código se añadirá a la sentencia. El hacker lo que busca es eliminar completamente nuestra base de datos. Para ello, cuando se le solicita el nombre del alumno, puede teclear:

```
xxx' OR '1' = '1
```

La cadena introducida por el usuario se complementa con la nuestra, resultando:

```
DELETE FROM Alumnos
WHERE nombre = 'xxx' OR '1' = '1'
```

Como la cadena «1» es siempre igual a «1», esto provocará que se eliminen todos los registros de la tabla Alumnos.

Actividad resuelta 14.7

Implementar un programa que solicite el nombre de un alumno y lo elimine. Aprovechar esta aplicación para practicar la técnica de SQL Injection.

Solución

```

Connection con;
Statement sentencia;
String sql;
String url = "jdbc:mysql://localhost/Instituto";
try {
    con = DriverManager.getConnection(url, "Pepe", "12345");
    sentencia = con.createStatement();

    //vamos pedir el nombre del alumno a eliminar:
    System.out.println("Nombre del alumno:");
    String nombre = new Scanner(System.in).nextLine();

    //Formamos la consulta
    sql = "DELETE FROM Alumnos WHERE nombre ='" + nombre + "'";
    System.out.println(sql); //mostramos la consulta SQL
    sentencia.executeUpdate(sql); //ejecutamos
    con.close(); //cerramos la conexión
} catch (SQLException ex) {
    System.out.println("Ha ocurrido algún error.");
}

```



Figura 14.11. La extraña posición de las comillas utilizadas en el código inyectado tiene como finalidad completar las comillas de la consulta de la aplicación y utilizar el código propio. En la condición sería más cómodo usar `1 = 1`, pero el hecho de que exista una comilla final en la consulta de la aplicación es lo que fuerza a la comparación `'1' = '1'`.

SQL Injection no solo funciona cuando nuestra aplicación elimina, también se puede inyectar código malicioso en otros casos. Por ejemplo, en una consulta que muestre todos los alumnos de un curso

```
sql = "SELECT * FROM Alumnos WHERE curso ='" + curso + "'";
```

si al pedir el curso al usuario, este teclea:

```
xxx'; DROP TABLE Alumnos; SELECT * FROM Alumnos WHERE curso = 'x
```

la cadena que se manda al SGBD para su ejecución estará compuesta de tres sentencias, siendo una de ellas la eliminación completa de una tabla

```

SELECT * FROM Alumnos WHERE curso = 'xxx'
DROP TABLE Alumnos
SELECT * FROM Alumnos WHERE curso = 'x'

```

Por suerte, el método `executeUpdate()` solo permite la ejecución de una única instrucción. Pero existen otros mecanismos de acceso a una base de datos en otros lenguajes donde el código anterior sí funcionaría.

La vulnerabilidad que explota SQL Injection es que no se validan los datos de entrada a la aplicación. Para solucionar esto tendríamos que validar todos y cada uno de los datos que se recogen en una aplicación, evitando que incluyan comillas simples, puntos y comas y todos aquellos caracteres que facilitan la inyección de código SQL.

Otra solución para evitar la inyección de código es el uso de sentencias parametrizadas.

14.7. Sentencias parametrizadas

Como se ha visto, para evitar SQL Injection no es buena idea construir sentencias como concatenación de cadenas a partir de datos introducidos por el usuario. En lugar de esto, usaremos sentencias parametrizadas, que son aquellas que incluyen unos marcadores o parámetros que se sustituyen por valores. Este mecanismo permite adaptar y reutilizar la misma consulta varias veces. En JDBC la interfaz `PreparedStatement` representa una consulta parametrizada.

Veamos el concepto de consulta parametrizada con un ejemplo: si deseo consultar a todos los alumnos de un curso concreto cuya nota media es superior a cierta nota de corte. La consulta parametrizada se formará:

```

SELECT *
FROM Alumnos
WHERE curso = ? AND
media > ?

```

En la consulta se usa el símbolo de cierre de interrogación (?) para introducir un parámetro. Si me interesa que el curso sea 1A y la nota de corte sea 6,0, puedo asignar estos valores a los parámetros:

- El primer parámetro: ? = 1A.
- El segundo parámetro: ? = 6.0

De esta forma la consulta queda:

```

SELECT *
FROM Alumnos
WHERE curso = '1A' AND
media > 6.0

```

Una ventaja de este tipo de consultas es que no es necesario prestar atención a las comillas, solo hay que asignar valores a los parámetros y es el propio JDBC lo que determina qué campos se entrecomillan.

En el caso de tener que reutilizar la consulta, es tan simple como asignar nuevos valores a los parámetros.

Veamos el código necesario para utilizar las consultas con parámetros:

```
//Consulta con parámetros. Cada parámetro se indica con ?
String sql = "SELECT nombre, media FROM Alumnos " +
    "WHERE curso = ? AND " +
    "        media > ?";

//Creamos un objeto de tipo PreparedStatement:
PreparedStatement sentencia = con.prepareStatement(sql);
//asignamos los parámetros:
sentencia.setString(1, curso); //el primer ? corresponde al curso
sentencia.setDouble(2, notaCorte); //el segundo ? corresponde a la nota de corte
ResultSet rs = sentencia.executeQuery(); //ejecutamos la consulta
... //trabajamos con los datos resultantes
```

Para asignar los parámetros disponemos de los métodos:

- `void setString(int índiceParámetro, String valor)`
- `void setInt(int índiceParámetro, int valor)`
- `void setDouble(int índiceParametro, double valor)`
- `void setBoolean(int índiceParametro, boolean valor)`
- `void setDate(int índiceParámetro, Date valor)`

En todos los métodos el primer parámetro indica el índice del parámetro de la consulta (los parámetros se comienzan a contar desde 1). Y el segundo parámetro es el valor que asignar. Existen tantos métodos como tipos de datos, los expuestos aquí son solo algunos.

Nota técnica

Consulta en la ayuda oficial de Java el resto de métodos que existen para asignar distintos tipos de valores a un parámetro de una consulta del tipo `PreparedStatement`.

Actividad resuelta 14.8

Escribir un programa que muestre todos los alumnos de un curso cuya nota es mayor que cierta nota de corte. Tanto el curso como la nota de corte serán introducidos por el usuario.

Solución

```
Connection con;
PreparedStatement sentencia;
String sql;

String url = "jdbc:mysql://localhost/Instituto";
try {
    con = DriverManager.getConnection(url, "Pepe", "12345");
    //Formamos la consulta
    sql = "SELECT nombre, media FROM Alumnos WHERE curso = ? AND media > ?";
```

```
sentencia = con.prepareStatement(sql);
//Vamos pedir el curso:
System.out.println("Curso:");
String curso = new Scanner(System.in).nextLine();
//Vamos pedir la nota de corte:
System.out.println("Nota de corte:");
Double notaCorte = new Scanner(System.in).nextDouble();

//asignamos los parámetros:
sentencia.setString(1, curso); //el primer ? corresponde al curso
sentencia.setDouble(2, notaCorte); //el segundo ? es la nota de corte

ResultSet rs = sentencia.executeQuery();
while(rs.next()) {
    System.out.println(rs.getString("nombre") + "\t" +
        rs.getDouble("media"));
}

con.close(); //cerramos la conexión
} catch (SQLException ex) {
    System.out.println("Ha ocurrido algún error.");
}
```

En ocasiones, puede ser interesante que el valor de un parámetro sea nulo. Para ello disponemos del método de `PreparedStatement`:

- `void setNull(int índiceParámetro, int tipoSQL)`: pone a nulo el parámetro indicado, que se trata como si fuera del tipo `tipoSQL`, y que será uno de los tipos definidos en `java.sql.Types`: `INTEGER`, `BOOLEAN`, `VARCHAR`, `DECIMAL`, etcétera.

Actividad resuelta 14.9

Diseñar una aplicación que muestre un informe de los alumnos, según sus notas: todos los alumnos cuya nota es Bien (nota entre 6 y 7) y todos los alumnos cuya nota es Notable (entre 7 y 9). Cada informe debe ordenarse por la nota de forma ascendente.

Para realizar esta actividad usaremos una consulta con parámetros y la reutilizaremos.

Solución

```
Connection con;
String url = "jdbc:mysql://localhost/Instituto";
try {
    con = DriverManager.getConnection(url, "Pepe", "12345");
    System.out.println("Alumnos con Bien:");
    alumnosXNota(con, 6, 7);
    System.out.println("\nAlumnos con Notable:");
    alumnosXNota(con, 7, 9);
    con.close(); //cerramos la conexión
} catch (SQLException ex) {
    System.out.println("Ha ocurrido algún error.");
}
```

El código correspondiente al método que muestra a los alumnos según sus notas es:

```
void alumnosXNota(Connection con, double n1, double n2) throws SQLException {
    //Formamos la consulta
    String sql = "SELECT nombre, media FROM Alumnos WHERE ?<=media AND media<? " +
        "ORDER BY media ASC";
    PreparedStatement sentencia = con.prepareStatement(sql);
    //asignamos los parámetros:
    sentencia.setDouble(1, n1); //nota mínima
    sentencia.setDouble(2, n2); //nota máxima
    ResultSet rs = sentencia.executeQuery();

    while (rs.next()) { //mostramos
        System.out.println(rs.getString("nombre") + "\t\t" +
            rs.getString("media"));
    }
}
```

14.8. Operaciones CRUD

CRUD son las siglas en inglés de «crear, leer, actualizar y borrar», y se usa para referirse a las operaciones básicas que se realizan en una base de datos.

Hasta ahora, hemos trabajado con datos aislados como simples variables. Esto nos ha servido para acercarnos a la API de JDBC y al manejo de una base de datos; pero esta forma de trabajar no es la más habitual. Ahora manejaremos clases y objetos (clase `Persona`, clase `Mascota`, etc.) a las que incorporaremos las operaciones CRUD, que se encargarán de gestionar el objeto en la base de datos.

Existe una técnica llamada *mapeo objeto-relacional*, que consiste en mapear o vincular cada atributo de una clase con un campo de una tabla en la base de datos. Es decir, convertimos los datos como objeto en un registro de una tabla relacional, y viceversa.

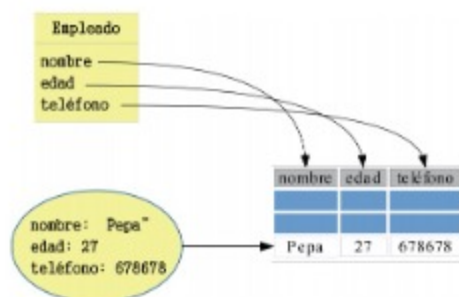


Figura 14.12. La clase `Empleado` se mapea en una tabla, cada atributo de la clase se almacena en un campo de la tabla. Los tipos deben coincidir o, al menos, ser compatibles. Cada objeto se almacena en un registro. Aquí los datos son los mismos, solo varía su representación: como objeto en el ámbito de la aplicación y como registro en el ámbito de la base de datos.

Recuerda



Las bases de datos que almacenan sus datos en tablas, formadas por columnas (campos) y filas (registros) reciben el nombre de **bases de datos relacionales**. Este nombre proviene del **modelo relacional**, que es el modelo de datos que utiliza este tipo de bases de datos.

El modelo relacional se lo debemos a Edgar Frank Codd.

A cada clase de nuestra aplicación se le añadirán los métodos:

- `create()`: inserta los datos del objeto en la base de datos mediante una sentencia INSERT.
- `read()`: mediante una sentencia SELECT, rescata los datos de la BD y los carga en un objeto.
- `update()`: actualiza los datos de un objeto (que se habrán modificado) guardándolos en la base de datos, mediante una sentencia UPDATE.
- `delete()`: elimina los datos del objeto actual de la base de datos.

Argot técnico



El flujo de información del método `update()` suele ser desde el objeto hacia la base de datos. Es decir, cuando sabemos que un objeto ya está insertado en la BD y se ha modificado, guardamos las actualizaciones en la BD mediante este método.

Puede que nos interese justo lo contrario: tras modificar un objeto, desechar estos cambios y que el objeto vuelva a cargarse con los datos existentes en la BD. Para ello, podemos utilizar el método `read()`. En la bibliografía se suele denominar `refresh()` al método que hace esto.

Actividad resuelta 14.10

Diseñar la clase `Alumno` que tiene los siguientes atributos, de manera que se pueda almacenar sus objetos en la base de datos Instituto:

- `id`: es el número identificativo único asignado a cada alumno, que sirve para distinguirlos de forma unívoca.
- `nombre`: del alumno. Su tamaño nunca superará los 30 caracteres. En caso de tener una longitud superior, se acortará.
- `fecNacimiento`: fecha de nacimiento del alumno.
- `notaMedia`: la calificación media del alumno en el curso.
- `curso`: cadena formada por dos caracteres que identifica el curso donde está matriculado el alumno.

Solución

```
public class Alumno {
    private final int TAM_NOMBRE = 30; //especifica el tamaño máximo del nombre
    //que en la BD se ha definido como un VARCHAR(30)
    private int id; //atributo identificador
```

```

private String nombre;
private Date fNacimiento;
private double notaMedia;
private String curso; //La cadena curso debe tener una longitud máx. de 2

/* Constructor que crea un objeto Alumno únicamente con su identificador.
 * Se utiliza para leer los datos desde la BD a partir de la clave. */
public Alumno(int id) {
    this.id = id;
}

public Alumno(int id, String nombre, Date fNacimiento, double notaMedia, String curso) {
    this.id = id;
    setNombre(nombre);
    this.fNacimiento = fNacimiento;
    this.notaMedia = notaMedia;
    this.setCurso(curso);
}

/* Crea una conexión con el SGBD y lo devuelve. La responsabilidad de cerrar
 * la conexión queda en manos de quién la use.*/
static private Connection conexion() {
    Connection con=null;
    String url = "jdbc:mysql://localhost/Instituto";
    try {
        con = DriverManager.getConnection(url, "Pepe", "12345");
    } catch (SQLException ex) {
        System.out.println("Error al conectar al SGBD.");
    }
    return con;
}

//Este método inserta el objeto this como un registro de la tabla Alumnos,
//en la BD Instituto. Debemos hacer coincidir el nombre de los atributos del
//objeto y el de los campos de la tabla.
public void create() {
    Connection con = conexion();
    String sql = "INSERT INTO Alumnos (num, nombre, fnac, media, curso) * +
                * VALUES ( ?, ?, ?, ?, ? )";
    try {
        PreparedStatement sentencia = con.prepareStatement(sql);
        sentencia.setInt(1, id);
        sentencia.setString(2, nombre);
        sentencia.setDate(3, new java.sql.Date(fNacimiento.getTime()));
        sentencia.setDouble(4, notaMedia);
        sentencia.setString(5, curso);
        sentencia.executeUpdate();
        con.close(); //cerramos la conexión
    } catch (SQLException ex) {
        System.out.println("Error al insertar.");
    }
}

//Escribiremos dos versiones de read():
//1. Será un método estático al que se le pasa la clave(id). Leerá los
//datos correspondiente a ese id, construirá un objeto con esos datos

```

```

// y devolverá el objeto creado.
//2. Será un método no estático que utilizará this.id, leerá los datos del
//registro correspondiente a esa clave, y asignará los valores de los
//campos en los atributos del propio objeto (this).
//Método estático: devuelve el objeto leído.
static public Alumno read(int id) {
    Alumno alumno = null;
    String sql = "SELECT * FROM Alumnos WHERE num = ?";
    try {
        Connection con = conexion();
        PreparedStatement sentencia = con.prepareStatement(sql);
        sentencia.setInt(1, id); //asignamos la clave a buscar
        ResultSet rs = sentencia.executeQuery();
        //Al estar buscando por la clave, solo existen dos alternativas:
        //La encuentra: el resultSet tendrá un único registro.
        //No la encuentra: el resultSet estará vacía.
        if (rs.next()) { //si hay un registro
            String nombre = rs.getString("nombre");
            Date fNacimiento = rs.getDate("fNac");
            Double notaMedia = rs.getDouble("media");
            String curso = rs.getString("curso");
            //creamos un objeto con los datos obtenidos
            alumno = new Alumno(id, nombre, fNacimiento, notaMedia, curso);
        }
    } catch (SQLException ex) {
        System.out.println("Error al consultar un alumno.");
    }
    return alumno; //si no encuentra el id devolverá null
}

//Método no estático: busca en la BD un registro con la misma clase que
//este objeto (this.id), lo lee y asigna estos valores a los atributos
//de this.
public void read() {
    String sql = "SELECT * FROM Alumnos WHERE num = ?";
    try {
        Connection con = conexion();
        PreparedStatement sentencia = con.prepareStatement(sql);
        sentencia.setInt(1, id); //asignamos la clave a buscar
        ResultSet rs = sentencia.executeQuery();

        //Al estar buscando por la clave, solo existen dos alternativas:
        //La encuentra: el resultSet tendrá un único registro.
        //No la encuentra: el resultSet estará vacía.
        if (rs.next()) { //si hay un registro
            this.nombre = rs.getString("nombre");
            this.fNacimiento = rs.getDate("fNac");
            this.notaMedia = rs.getDouble("media");
            this.curso = rs.getString("curso");
        }
    } catch (SQLException ex) {
        System.out.println("Error al consultar un alumno.");
    }
}

//Actualiza los valores del objeto this en la BD.
//Supondremos que el objeto ya dispone de su registro.

```

```
//Es habitual, no permitir que se modifique el identificador de un objeto.
public void update() {
    String sql = "UPDATE Alumnos SET nombre=?, fNac=?, media=?, curso=? WHERE num = ?";
    try {
        Connection con = conexion();
        PreparedStatement sentencia = con.prepareStatement(sql);
        sentencia.setString(1, this.nombre);
        sentencia.setDate(2, (java.sql.Date)this.fNacimiento);
        sentencia.setDouble(3, this.notaMedia);
        sentencia.setString(4, this.curso);
        sentencia.setInt(5, id); //asignamos la clave a buscar
        sentencia.executeUpdate();
    } catch (SQLException ex) {
        System.out.println("Error al actualizar un alumno.");
    }
}

//Eliminamos el registro correspondiente al objeto this de la BD.
public void delete() {
    String sql = "DELETE FROM Alumnos WHERE num = ?";
    try {
        Connection con = conexion();
        PreparedStatement sentencia = con.prepareStatement(sql);
        sentencia.setInt(1, id); //asignamos la clave a buscar
        sentencia.executeUpdate();
    } catch (SQLException ex) {
        System.out.println("Error al eliminar un alumno.");
    }
}

public int getId() { return id; }

public void setId(int id) { this.id = id; }

public String getNombre() { return nombre; }

//Limita la longitud del nombre al número de caracteres indicado por la
//constante TAM_NOMBRE.
public void setNombre(String nombre) {
    this.nombre = nombre.substring(0, Math.min(TAM_NOMBRE, nombre.length()));
}

public Date getfNacimiento() { return fNacimiento; }

public void setfNacimiento(Date fNacimiento) { this.fNacimiento = fNacimiento; }

public double getNotaMedia() { return notaMedia; }

public void setNotaMedia(double notaMedia) { this.notaMedia = notaMedia; }

public String getCurso() { return curso; }

//en la BD es un VARCHAR(2), tenemos que limitar el String a un máximo de 2 caracteres.
public void setCurso(String curso) {
    this.curso = curso.substring(0, Math.min(2, nombre.length()));
}
```

```
@Override
public String toString() {
    return "Alumno{id = " + id + ", nombre = " + nombre + ", fNacimiento = " +
        fNacimiento + ", notaMedia = " + notaMedia + ", curso = " + curso + "'}";
}
```

14.9. Objeto de acceso a datos

Desde el punto de vista de la abstracción en la POO, un enfoque purista defenderá que una clase debe plasmar sus características y comportamientos esenciales. En el caso de un alumno, habrá que manejar sus datos, sus notas, sus matrículas, etc. Pero estaremos de acuerdo en que cualquier operación CRUD en una BD no pertenece al mundo de un alumno.

El hecho de añadir los métodos relacionados con las operaciones de la BD a una clase como **Alumno** es una distorsión de la abstracción que se hace de la realidad. Por este motivo, existe un enfoque en el que cada clase de interés en nuestra aplicación se diseñará única y exclusivamente atendiendo a la abstracción del mundo real. Y si necesitamos que esa clase se almacene en una BD, diseñaremos una segunda clase, denominada DAO (por sus siglas en inglés de «objeto de acceso a datos») con la única misión de realizar las operaciones CRUD en la BD para su correspondiente clase.

Alumno	AlumnoDAO
-id: int	+create(alumno: Alumno)
-nombre: String	+read(id: int): Alumno
-fNacimiento: Date	+update(alumno: Alumno)
-notaMedia: double	+delete(id: int)
-curso: String	-conectar(): Connection

Figura 14.13. Clase **Alumno**, que solo contiene atributos y métodos producto de la abstracción de los alumnos en la realidad; y clase **AlumnoDAO**, que está diseñada para interactuar con objeto de tipo **Alumno** con la BD.

Cada clase DAO tendrá los mismos métodos, pero estará diseñada específicamente para una clase del modelo de dominio.

Recuerda

Un modelo de dominio es una abstracción del sistema que modelar. Las clases del modelo de dominio son aquellas que abstraen esta realidad. Si, por ejemplo, deseo informatizar una ferretería, las clases del modelo de dominio serán aquellas que representan a los clientes, productos, facturas y pedidos. No pertenecerán al modelo de dominio cualquier clase auxiliar, las clases de la interfaz gráfica ni las clases de JDBC.



Actividad resuelta 14.11

Escribir la clase `AlumnoDAO` que se encargará de gestionar los objetos de la clase `Alumno` en la BD. La clase `AlumnoDAO` tendrá los métodos representados en el diagrama de clases de la Figura 14.13.

Solución

```
public class AlumnoDAO {
    //Crea una conexión con el SGHD y la devuelve.
    private static Connection conectar() {
        Connection con = null;
        String url = "jdbc:mysql://localhost/Instituto";
        try {
            con = DriverManager.getConnection(url, "Pepe", "12345");
        } catch (SQLException ex) {
            System.out.println("Error al conectar al SGHD.");
        }
        return con;
    }

    //Este método inserta el Alumno pasado como parámetro como un registro de la
    //tabla Alumnos, en la BD Instituto. Debemos hacer coincidir el nombre de
    //los atributos del objeto y el de los campos de la tabla.
    public static void create(Alumno alumno) {
        //Si el alumno pasado es nulo no haremos nada.
        if (alumno != null) {
            Connection conexion = conectar();
            String sql = "INSERT INTO Alumnos (num, nombre, fnac, media, curso) " +
                "VALUES ( ?, ?, ?, ?, ? )";
            try {
                PreparedStatement sentencia = conexion.prepareStatement(sql);
                sentencia.setInt(1, alumno.getId());
                sentencia.setString(2, alumno.getNombre());
                sentencia.setDate(3, new java.sql.Date(alumno.getfNacimiento().getTime()));
                sentencia.setDouble(4, alumno.getNotaMedia());
                sentencia.setString(5, alumno.getCurso());
                sentencia.executeUpdate();
                conexion.close(); //cerramos la conexión
            } catch (SQLException ex) {
                System.out.println("Error al insertar.");
            }
        }
    }

    //Lee los datos del alumno con clave id, construye un objeto Alumno con sus
    //datos y lo devuelve.
    public static Alumno read(int id) {
        Alumno alumno = null;
        String sql = "SELECT * FROM Alumnos WHERE num = ?";
        try {
            Connection conexion = conectar();
            PreparedStatement sentencia = conexion.prepareStatement(sql);
            sentencia.setInt(1, id); //asignamos la clave a buscar
            ResultSet rs = sentencia.executeQuery();
```

```
//Al estar buscando por la clave, solo existen dos alternativas:
//1. La encuentra: el resultSet tendrá un único registro.
//2. No la encuentra: el resultSet estará vacío.
if (rs.next()) { //si hay un registro
    String nombre = rs.getString("nombre");
    Date fNacimiento = rs.getDate("fNac");
    Double notaMedia = rs.getDouble("media");
    String curso = rs.getString("curso");
    //creamos un objeto con los datos obtenidos
    alumno = new Alumno(id, nombre, fNacimiento, notaMedia, curso);
    conexion.close();
}
```

```
} catch (SQLException ex) {
    System.out.println("Error al consultar un alumno.");
}
return alumno; //si no encuentra el id devolverá null
}
```

```
//Actualiza los valores del objeto alumno pasado como parámetro en la BD.
//Supondremos que el objeto ya dispone de su registro.
//No permitimos que se modifique el identificador de un objeto.
public static void update(Alumno alumno) {
```

```
    if (alumno != null) {
        String sql = "UPDATE Alumnos SET nombre=?, fNac=?, media=?, " +
            "curso=? WHERE num=?";
        try {
            Connection conexion = conectar();
            PreparedStatement sentencia = conexion.prepareStatement(sql);
            sentencia.setString(1, alumno.getNombre());
            sentencia.setDate(2, (java.sql.Date) alumno.getfNacimiento());
            sentencia.setDouble(3, alumno.getNotaMedia());
            sentencia.setString(4, alumno.getCurso());
            sentencia.setInt(5, alumno.getId()); //asignamos la clave a buscar
            sentencia.executeUpdate();
            conexion.close();
        } catch (SQLException ex) {
            System.out.println("Error al actualizar un alumno.");
        }
    }
}
```

```
//Eliminamos el registro correspondiente al registro con clave id.
```

```
public static void delete(int id) {
    String sql = "DELETE FROM Alumnos WHERE num = ?";
    try {
        Connection conexion = conectar();
        PreparedStatement sentencia = conexion.prepareStatement(sql);
        sentencia.setInt(1, id); //asignamos la clave a buscar
        sentencia.executeUpdate();
        conexion.close();
    } catch (SQLException ex) {
        System.out.println("Error al eliminar un alumno.");
    }
}
```

En esta actividad se ha implementado la clase `AlumnoDAO` con todos los métodos estáticos. Con respecto a la conexión, se ha mantenido la política de:

1. Realizar una conexión con la BD.
2. Realizar una operación (create, update, delete o insert).
3. Cerrar la conexión.

Actividad resuelta 14.12

Rediseñar la clase `AlumnoDAO` para que permita una política distinta con respecto a la conexión a la BD:

1. Conectamos con la BD al principio.
2. Se realizan multitud de operaciones.
3. Se cierra la conexión con la BD.

Para implementar este enfoque debemos mantener un atributo con la conexión, que se abrirá en cuanto un objeto `AlumnoDAO` se construya. Ahora los métodos no podrán ser estáticos, ya que requieren el uso de un atributo del objeto (la conexión).

Podemos aprovechar el enfoque no estático para definir unas constantes que sirvan para configurar los parámetros de la conexión a la BD.

Solución

La clase `AlumnoDAO` con sus atributos y constructor quedaría:

```
public class AlumnoDAO {
    private Connection conexion;
    private final String USUARIO = "Pepe";
    private final String PASSWORD = "12345";
    private final String MAQUINA = "localhost";
    private final String BD = "Instituto";

    //En el constructor creamos la conexión, que se mantendrá abierta todo el tiempo.
    public AlumnoDAO() {
        //establecemos la conexión con la BD:
        conexion = conectar();
    }

    //Crea una conexión con el SGBD y la devuelve.
    private Connection conectar() {
        Connection con = null;
        String url = "jdbc:mysql://" + MAQUINA + "/" + BD;
        try {
            con = DriverManager.getConnection(url, USUARIO, PASSWORD);
        } catch (SQLException ex) {
            System.out.println("Error al conectar al SGBD.");
        }
        return con;
    }

    //resto de métodos
    //...
}
```

El resto de los métodos son prácticamente idénticos, con la salvedad de que no se declaran estáticos y no construirán la conexión ni la cerrarán; para ello usarán la conexión disponible en la clase. A modo de ejemplo pondremos el método `delete()`:

```
//Eliminamos el registro correspondiente al registro con clave id.
public void delete(int id) {
    String sql = "DELETE FROM Alumnos WHERE num = ?";
    try {
        PreparedStatement sentencia = conexion.prepareStatement(sql);
        sentencia.setInt(1, id); //asignamos la clave a buscar
        sentencia.executeUpdate();
    } catch (SQLException ex) {
        System.out.println("Error al eliminar un alumno.");
    }
}
```

A las clases DAO podemos añadirles tantos métodos como necesitemos. Es habitual disponer de un método que devuelva una lista (`java.util.List`) con tantos objetos como registros tenga la tabla correspondiente. También es usual encontrar en la clase DAO métodos que devuelven todos los objetos que cumplen que cierto atributo coincide con un valor determinado.

Actividad resuelta 14.13

Añadir un método a la clase `AlumnoDAO` que devuelva una lista con todos los alumnos que existen en la base de datos.

Solución

```
//Construye una lista con todos los alumnos que existen en la BD
List<Alumno> todosAlumnos() {
    List<Alumno> todos = new LinkedList<>();
    Alumno alumno = null;
    String sql = "SELECT * FROM Alumnos";
    try {
        Statement sentencia = conexion.createStatement();
        ResultSet rs = sentencia.executeQuery(sql);
        //Recorremos el resultado y crearemos un objeto por cada registro
        while (rs.next()) {
            int id = rs.getInt("num");
            String nombre = rs.getString("nombre");
            Date fNacimiento = rs.getDate("fNac");
            Double notaMedia = rs.getDouble("media");
            String curso = rs.getString("curso");
            //creamos un objeto con los datos obtenidos
            alumno = new Alumno(id, nombre, fNacimiento, notaMedia, curso);
            // y lo añadimos a la lista
            todos.add(alumno);
        }
    } catch (SQLException ex) {
        System.out.println("Error al consultar un alumno.");
    }
    return todos;
}
```

14. CONEXIÓN A BASE DE DATOS: JDBC

INFORMÁTICA Y COMUNICACIONES

Dependiendo de la política de conexión que usemos en la implementación de la clase DAO, sus métodos serán no estáticos (como en este caso) o se definirán con `static` si cada método se encarga de abrir y cerrar su propia conexión.

Actividad resuelta 14.14

Incluir en la clase `AlumnoDAO` un método que permita buscar a todos los alumnos que comparten curso.

Solución

```
/*Construye una lista con todos los alumnos que pertenecen al mismo curso.
En esta caso estamos implementando un método que realiza la búsqueda por
uno de los atributos (curso) de la clase Alumno. */
List<Alumno> alumnosXCurso(String curso) {
    List<Alumno> mismoCurso = new LinkedList<>();
    Alumno alumno = null;
    String sql = "SELECT * FROM Alumnos WHERE curso = ?";
    try {
        PreparedStatement sentencia = conexion.prepareStatement(sql);
        sentencia.setString(1, curso);

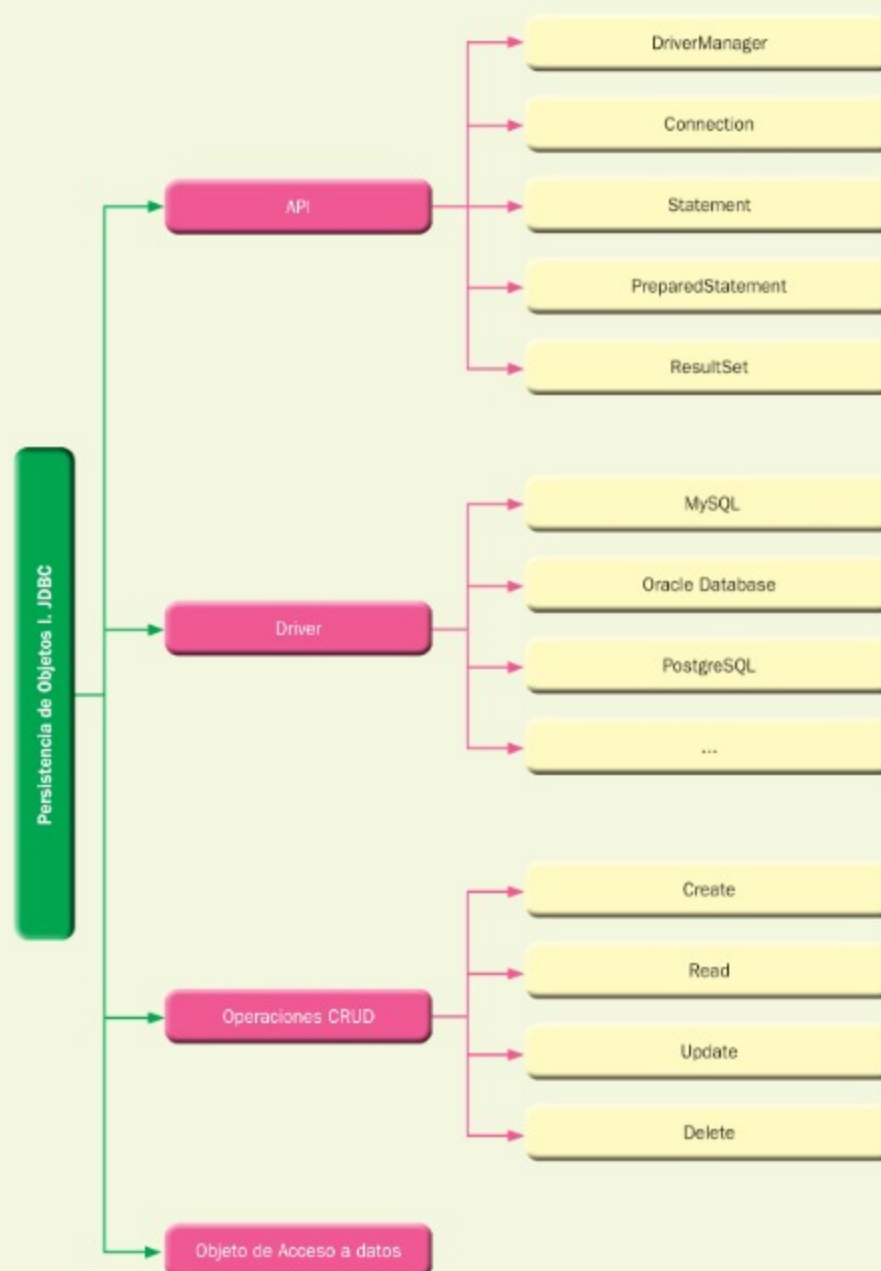
        ResultSet rs = sentencia.executeQuery();
        //Recorremos el resultado y crearemos un objeto por cada registro
        while (rs.next()) {
            int id = rs.getInt("num");
            String nombre = rs.getString("nombre");
            Date fNacimiento = rs.getDate("fNac");
            Double notaMedia = rs.getDouble("media");
            //creamos un objeto con los datos obtenidos
            alumno = new Alumno(id, nombre, fNacimiento, notaMedia, curso);
            mismoCurso.add(alumno); // y lo añadimos a la lista
        }
    } catch (SQLException ex) {
        System.out.println("Error al consultar un alumno.");
    }
    return mismoCurso;
}
```

Enlaces web de interés

- **MySQL Community Downloads** - dev.mysql.com/downloads/connector/j/
(Lugar de descarga del driver JDBC de MySQL/MariaDB)
- **JDBC and UCP Downloads page** - www.oracle.com/database/technologies/appdev/jdbc-downloads.html
(Página de descarga del driver JDBC de Oracle)
- **PostgreSQL JDBC Driver** - jdbc.postgresql.org
(Página de descarga del driver JDBC de PostgreSQL)

14. CONEXIÓN A BASE DE DATOS: JDBC

MAPA CONCEPTUAL



Actividades de comprobación

- 14.1.** Las clases que componen la API de JDBC se encuentran localizadas en el paquete:
- `java.mysql.api`
 - `java.util.sql`
 - `java.sql`
 - `java.mysql.jdbc`
- 14.2.** Las siglas URL se corresponden con:
- Unificator Remote Locator.
 - Unique Remote Locator.
 - Uniform Resource Locator.
 - Uniform Remote Locator.
- 14.3.** Señala la afirmación correcta:
- La clase Statement permite la ejecución de sentencias SELECT.
 - La clase Statement permite la ejecución de sentencias INSERT.
 - La clase Statement permite la ejecución de sentencias UPDATE.
 - La clase Statement permite la ejecución de sentencias DELETE.
- 14.4.** SQL Injection es una técnica utilizada habitualmente para:
- Insertar grandes cantidades de datos en la BD.
 - Hackear aplicaciones que utilizan JDBC.
 - Realizar comprobaciones de integridad en un SGBD.
 - Todas las respuestas son ciertas.
- 14.5.** El formato de localizador de recurso que permite crear una conexión con un SGBD MySQL tiene el formato:
- `jdbc:mysql://<máquina>/<BD>`
 - `jdbc:<máquina>://mysql/<BD>`
 - `mysql:jdbc://<máquina>/<BD>`
 - `jdbc:<BD>://<máquina>/mysql`
- 14.6.** Cuando estamos utilizando los resultados de una consulta representados en una clase `ResultSet`, podremos:
- Acceder a todos los registros simultáneamente y a todos los campos.
 - Acceder a un único registro simultáneamente y a todos sus campos.
 - Acceder a todos los registros simultáneamente, pero solo a un campo.
 - Todas las respuestas son ciertas.
- 14.7.** En una consulta parametrizada, la forma de indicar a qué parámetro queremos asignarle un valor puede ser:
- Siempre por el nombre del parámetro.
 - Podemos elegir entre el nombre o la posición que ocupa el parámetro.
 - Siempre por la posición que ocupa el parámetro.
 - Para configurar los parámetros utilizaremos siempre la clase `Parameter`.

- 14.8.** El acrónimo CRUD hace referencia a un conjunto de operaciones que incluyen:
- Consultas y movimientos de datos.
 - Inserción, consultas, extracción, lectura y creación de datos.
 - Borrado, eliminación, actualización, consulta y copias de seguridad de los datos.
 - Eliminación, inserción, actualización y consulta de datos.
- 14.9.** La técnica de mapeo objeto-relacional hace referencia a:
- Tratar los objetos de una aplicación y las relaciones (asociaciones) que existen entre ellos.
 - Dibujar un mapa o diagrama que muestre todas las clases y objetos que intervienen en una aplicación.
 - Convertir cada atributo de un objeto en un campo de un registro en una base de datos relacional y viceversa.
 - Tratar todos los mapas relacionales como si fueran objetos de una clase DAO.
- 14.10.** ¿Cuál es la ventaja de implementar las operaciones CRUD dentro de una clase DAO con respecto a implementar las operaciones DAO en una clase del modelo de dominio?
- Distribuye la lógica del programa entre clases con responsabilidades diferentes.
 - No modifica la representación (abstracción) de las clases del modelo de negocio.
 - Al limitar el código implementado en distintas clases, facilita el mantenimiento de la aplicación.
 - Todas las respuestas son ciertas.

Actividades de aplicación

Disponemos de la base de datos Empresa donde se almacena la información de los empleados de una compañía y las oficinas de las que dispone en distintas ciudades. La base de datos está compuesta por las tablas:

Empleados, con los campos:

- `numemp`: número identificativo del empleado.
- `nombre`.
- `edad`.
- `oficina`: clave foránea de la tabla Oficinas, que determina la oficina en la que el empleado trabaja.
- `puesto`: empleo que desarrolla en la empresa.
- `contrato`: fecha en la que se contrató al empleado.

Oficinas, con los campos:

- `oficina`: número identificativo de cada oficina.
- `ciudad`.
- `superficie`: extensión en metros cuadrados de la oficina.
- `ventas`: importe de las ventas de esta oficina.

Utilizando la base de datos Empresa, se pide realizar las siguientes actividades:

ACTIVIDADES FINALES

14. CONEXIÓN A BASE DE DATOS: JDBC

- 14.11. Crea un programa que muestre todos los empleados.
- 14.12. Utiliza la técnica de mapeo objeto-relacional para cargar todas las oficinas de la base de datos en una lista de oficinas. Muestra el contenido de la lista de objetos creada.
- 14.13. Modifica la Actividad de aplicación 14.12 para que la aplicación solicite al usuario el nombre de una ciudad, y que muestre las oficinas ubicadas en dicha ciudad.
- 14.14. Muestra un listado con el nombre y la edad de los empleados cuya edad se encuentra comprendido entre unos valores máximos y mínimos que introducirá el usuario.
- 14.15. Crea un programa que lea todos los datos de un empleado, excepto la fecha de contratación que será la de hoy, y los inserte en la base de datos.
- 14.16. Modifica la Actividad de aplicación 14.15 para que el programa utilice mapeo objeto-relacional. Es decir, hemos de insertar los datos de un empleado creando previamente un objeto de tipo `Empleado`. Utiliza también un objeto DAO para la inserción. No será necesario implementar otros métodos del objeto DAO.
- 14.17. Añade el código necesario a la Actividad de aplicación 14.16 para comprobar que el empleado que se va a insertar no existe en la base de datos. Para ello es necesario controlar que no encontramos un empleado con idéntico «numemp» en la base de datos.
- 14.18. Repite la Actividad de aplicación 14.17, pero controlando que existe el número de oficina introducido. Utiliza también una clase DAO para las oficinas.
- 14.19. Escribe una aplicación que cambie a todos los empleados que trabajan en una oficina a otra. Muestra a los empleados afectados por el cambio de oficina, antes y después de la modificación.
- 14.20. Solicita por teclado el número de un empleado y bórralo de la base de datos. Implementa la solución como parte de la clase DAO para empleados.
- 14.21. Añade a la clase `OficinaDAO` un método que devuelva una lista con todas las oficinas. Prueba el método mostrando todas las oficinas existentes en la base de datos.
- 14.22. Aprovecha el método de la Actividad de aplicación 14.21 para mostrar las oficinas cuya superficie es superior a una extensión introducida por el usuario.
- 14.23. Crea un programa que permita modificar la ciudad e incrementar las ventas de distintas oficinas.

■ Actividades de ampliación

- 14.24. Swing es una biblioteca gráfica para Java que permite crear ventanas que contienen distintos elementos gráficos como botones, cajas de texto, listas desplegables, etc. Se pide investigar sobre Swing y añadir una interfaz gráfica sencilla que permita introducir los datos de una oficina mediante un formulario y guardar dichos datos en la base de datos.

14. CONEXIÓN A BASE DE DATOS: JDBC

ACTIVIDADES FINALES

- 14.25. Construye una interfaz gráfica con Swing que, mediante el uso de una tabla (clase `JTable`), muestre a todos los empleados que existen en la base de datos.
- 14.26. La API de JDBC de Java se considera de bajo nivel, lo que provoca que sea el programador el encargado de la gestión de los datos en la aplicación.

Existe una API de más alto nivel (que internamente usa JDBC) denominada JPA y que se encarga de la gestión de los objetos de nuestra aplicación en la base de datos. Se pide investigar sobre JPA: cuáles son sus funcionalidades, el sistema de anotaciones y qué es una entidad.
- 14.27. JPA dispone de su propio lenguaje de consulta de datos denominado JPQL. Es un lenguaje muy parecido a SQL y comparten gran parte de la sintaxis. Busca información sobre JPQL y sus distintas operaciones, y realiza una comparativa entre las mismas operaciones entre SQL y JPQL.
- 14.28. Java Reflection API es una librería que permite obtener información y manipular clases e interfaces, así como sus métodos y campos, en tiempo de ejecución. No es necesario (en tiempo de compilación) tener conocimiento de las clases con las que vamos a trabajar. Realizad por grupos una investigación sobre esta API y averigüad cuáles son sus principales funcionalidades.
- 14.29. Con JDBC tenemos que ser nosotros los encargados de crear las tablas en la base de datos antes de poder realizar operaciones con los datos. Java Reflection API puede ser útil para crear de forma automática una sentencia `CREATE TABLE` (que se ejecutará con JDBC) a partir de una clase dada. Se pide construir un método que reciba como parámetro una clase y construirá una tabla adecuada para almacenar los datos de los objetos de dicha clase.
- 14.30. Indaga sobre los mecanismos que utilizan otros lenguajes de programación para acceder a los datos almacenados en un SGBD relacional. Realiza una clasificación por lenguajes de programación según sean el tipo de mecanismo que usa.